HOCHSCHULE RAVENSBURG-WEINGARTEN UNIVERSITY OF APPLIED SCIENCES

# Lab Task 2: Programming the ET1 Robot

## Motor driver, debouncing, driving sequences and an emergency stop – in C, MicroPython or Rust

Embedded Computing – Master of Engineering – Lab

## Overview

You program the ET1 robot of the lab: a START button, a red LED and two DC motors behind an H-bridge. The robot is a model in the Wokwi simulator, so you need no hardware. A Raspberry Pi Pico controls it. Choose one language: **C**, **MicroPython** or **Rust**. Each language folder holds the same frame: wiring, set-up and empty functions with numbered `TODO` comments. You only write code, there are no questions to answer.

**Organisation.** About 4 hours, in teams of two. Hand in your source file and, for every task, a screenshot of the robot display with the track and of the serial monitor.

**Material.** The folder `Lab/2-programming/` of the lecture repository. Its `README.md` repeats the set-up steps.

## 1 Set-up

1. Install VS Code and the extension *Wokwi Simulator*. Get the free licence with F1 → *Wokwi: Request a New License*.
2. Open `Lab/2-programming/et1.code-workspace` (*File* → *Open Workspace from File*). VS Code then offers the other extensions it recommends.
3. Install the tools for your language:

| Language | Needs | Build |
|---|---|---|
| C | extension *PlatformIO IDE*; the first build downloads the Pico core (a few minutes) | `pio run` |
| MicroPython | Python 3; run the task *Python: set up .venv (once)* | `python make_uf2.py` |
| Rust | `rustup target add thumbv6m-none-eabi`, extension *rust-analyzer* | `cargo build --release` |

4. Build with *Terminal* → *Run Build Task*. Open a file of the same folder and start the simulation with F1 → *Wokwi: Start Simulator*. Your output and the robot's messages ("ET1 stopped at …") appear in the serial monitor.

After every change: build again, then restart the simulator. (MicroPython has a faster way: while the simulation runs, the task *Python: run main.py in the running simulation* sends `main.py` to it.)

**The simulated hardware**

| Pico pin | Connected to | |
|---|---|---|
| GP7 | red LED | high = on |
| GP5 | START button to GND | pull-up: low = pressed |
| GP18 / 17 / 16 | ENA / IN1 / IN2 | left motor |
| GP19 / 20 / 21 | ENB / IN3 / IN4 | right motor |
| GP0 / GP1 | serial monitor | C, Rust (Python: USB) |

**H-bridge:** `IN1 IN2` = `1 0` forward, `0 1` backward, `0 0` brake. The speed is the PWM duty cycle on ENA (right motor: IN3, IN4, ENB). Up to $12\,\%$ duty cycle a motor does not turn. Top speed $0.25$ m/s, wheel distance $14$ cm.

**Display of the ET1 chip:** a $2$ m $\times$ $2$ m floor (grid $0.5$ m), the robot (the white line points forward) and its track. Start: centre, facing up. `X`, `Y`: position in m; `H`: heading in degrees (+ = turned left); `D`: distance from the start; `L`, `R`: wheel speeds in %. Click the chip to set "Left motor weaker by (%)".




The ET1 display at the start

## 2 Tasks

Work through the `TODO`s in the order of their numbers. At the top of the file, set `TASK` to the task you are working on; the main loop then runs that task again and again.

### Task 1 Motor driver – `motor(side, speed)`

Write the driver for one motor: `speed` goes from $-100$ (full speed backward) via 0 (brake) to $+100$ (full speed forward).

a) Limit `speed` to $-100 \ldots 100$.

b) Select the PWM output and the two direction pins of the requested side.

c) Set the direction pins from the sign of `speed`.

d) Write $|$`speed`$|$ as duty cycle to the PWM output.

You test the driver in tasks 3 and 4: `L` and `R` on the display show what the wheels do.

### Task 2 Button and LED – `button_clicked()`, `task2()`

a) `button_clicked()` returns *true* exactly once per press, at the change from released to pressed. Keep the previous state between calls.

b) The contacts bounce: ignore every change that comes less than $30$ ms after the last accepted one.

c) `task2()`: every click toggles the LED and prints "LED on" or "LED off".

**Done when**

Each click toggles the LED once. Holding the button down does not toggle it again.

### Task 3 Turning – `task3()`

After a click: LED on. Only the left wheel drives forward at $50\,\%$ for $10\,\mathrm{s}$; the right wheel brakes. Then only the right wheel drives for $10\,\mathrm{s}$. Then stop, LED off, and print how long the sequence took.

**Done when**

The track shows two circles, one around each wheel, and the printed time is $20\,\mathrm{s}$.

### Task 4 Accelerate and brake – `ramp(dir)`, `task4()`

a) `ramp(dir)`: speed 10, 20, ..., $80\,\%$, each step for $0.5\,\mathrm{s}$, then back down 70, 60, ..., $0\,\%$, each for $0.5\,\mathrm{s}$. `dir` $= +1$ drives forward, $-1$ backward.

b) `task4()`: after a click, LED on, `ramp(+1)`, a pause of $1\,\mathrm{s}$, `ramp(-1)`, LED off.

c) Real motors are never equal. Set "Left motor weaker by" on the ET1 chip to $10\,\%$: ET1 now drives a curve. In `motor()`, add a trim factor for the left side (a constant at the top of the file) and find its value by trying.

**Done when**

Without the weaker motor, ET1 ends where it started: `D` shows 0.00. With "Left motor weaker by" at $10\,\%$ and your trim factor, `H` stays within $\pm 5^\circ$ at the far end of `ramp(+1)`.

### Task 5 Emergency stop – `wait_ms(ms)`

a) Replace the plain delay in `wait_ms()` by a loop that runs for `ms` milliseconds and calls `button_clicked()` every millisecond. On a click it stops the motors, prints "EMERGENCY STOP" and returns *false* at once; otherwise it returns *true*.

b) Use `wait_ms()` for every wait in tasks 3 and 4. When it returns *false*, end the whole sequence at once (motors off, LED off).

**Done when**

A second click while ET1 drives (task 3 or 4) stops the robot immediately and prints "EMERGENCY STOP". The next click starts the sequence again from the beginning.